I. Complexity Analysis (The Theoretical Foundation)

Q: Why do we treat Asymptotic Analysis (Big O) as an upper bound rather than a precise measurement?

Theory: Asymptotic analysis abstracts away hardware details (CPU speed, memory access time) to measure how an algorithm's resource usage scales relative to input size (1n).2

- **Big O (O):** Describes the **worst-case scenario** (upper bound).[3] It guarantees the algorithm will not perform worse than this function.
- **Big Omega (Ω):** Describes the **best-case scenario** (lower bound).[4]
- **Big Theta (Θ):** Describes the **tight bound** (average case) where the upper and lower bounds converge.[5]
- **Practical Nuance:** We often drop constants (e.g., $O(2n)$ becomes $O(n)$) because as $n \to \infty$, the growth rate dominates the constant factors. However, for small n, constants matter (e.g., Merge Sort vs. Insertion Sort).

Q: What is the difference between Amortized Time and Worst-Case Time?

Theory: Amortized analysis considers the average cost of an operation over a sequence of operations, rather than a single event.

- **Example (Dynamic Array / ArrayList):** Resizing an array takes $O(n)$ time because all elements must be copied to a new, larger memory block. However, this happens rarely (only when capacity fills). Most insertions are $O(1)$.
- **Math:** If resizing doubles capacity, the cost is distributed such that the *average* insertion is $O(1)$, even though the *worst-case* single insertion is $O(n)$.

II. Linear Data Structures

Q: Theoretically, why is a Linked List preferred over an Array for dynamic operations, despite poor cache locality?

Theory:

- **Arrays:** Contiguous memory blocks.[6]
    - *Pro:* $O(1)$ random access via pointer arithmetic (base_address + index * size). Excellent **spatial locality** (CPU cache hits).

    - *Con:* Insertion/Deletion is [7]$O(n)$ because elements must shift physically in memory.[8] Fixed size (static arrays) or expensive resizing (dynamic arrays).

- **Linked Lists:** Dispersed nodes connected by pointers.
    - *Pro:* Insertion/Deletion is $O(1)$ *if* you have the pointer to the location (no shifting required). Dynamic size allocation.
    - *Con:* $O(n)$ access time (must traverse). Poor **spatial locality** leads to frequent CPU cache misses, making them practically slower than arrays for small datasets.

Q: How does a Stack implement recursion at the hardware/OS level?

Theory: The Call Stack tracks active subroutines.

1. **Push:** When a function is called, a "stack frame" is pushed.[9] It contains the return address, local variables, and parameters.
2. **Pop:** When the function returns, the frame is popped, and control jumps back to the return address.
3. **Stack Overflow:** Occurs when the recursion depth exceeds the allocated stack memory space (infinite recursion or deep DFS).

III. Hashing

Q: What is the "Pigeonhole Principle" in the context of Hash Collisions?

Theory: A hash function maps a large keyspace (infinite possibilities) to a smaller index space (finite array slots).

- **Principle:** If you have N slots and $N+1$ keys, at least one slot must contain two keys. Collisions are theoretically inevitable.
- **Resolution Strategies:**
    1. **Chaining:** Each bucket points to a Linked List (or Balanced BST in Java 8+). Worst case becomes $O(n)$ (or $O(\log n)$ with BST) if all keys hash to the same slot.
    2. **Open Addressing:** If a collision occurs, find the next open slot using a probe sequence (Linear, Quadratic, or Double Hashing).

IV. Tree Structures

Q: Why is a Binary Search Tree (BST) not sufficient? Why do we need AVL or Red-Black Trees?

Theory: The efficiency of a BST depends entirely on its height (h).

- **Ideal Case:** Balanced tree, $h = \log_2 n$. Search is [10]$O(\log n)$.[11]
- **Worst Case (Skewed):** Insert sorted data (1, 2, 3, 4, 5). The tree becomes a Linked List. $h = n$. Search becomes $O(n)$.
- **Solution (Self-Balancing Trees):** AVL and Red-Black trees enforce **invariants** (rules) during insertion/deletion. They perform **rotations** to keep the height strictly $O(\log n)$, guaranteeing logarithmic performance.

Q: How does a Heap (Priority Queue) differ from a BST?

Theory:

- **Structure:** A Heap is a "Complete Binary Tree" (filled left-to-right), usually implemented as an array. A BST is a strictly ordered node structure.
- **Ordering:**
    - *BST:* Left Child < Parent < Right Child. (Used for Searching).
    - *Max-Heap:* Parent [12]$\ge$ Children.[13] (Used for finding Max/Min quickly). There is no relationship between left and right siblings.
- **Performance:** Finding max is $O(1)$ in a Max-Heap but $O(\log n)$ (or $O(n)$ if unbalanced) in a BST.

V. Graphs

Q: When should you theoretically use an Adjacency Matrix vs. an Adjacency List?

Theory:

- **Adjacency Matrix ($V \times V$ 2D Array):**
    - Space: $O(V^2)$.
    - Query (u, v connected?): $O(1)$.
    - Use Case: **Dense graphs** (where edges $E \approx V^2$).

- **Adjacency List (Array of Lists):**
    - Space: $O(V + E)$.
    - Query: $O(\text{degree of } u)$.
    - Use Case: **Sparse graphs** (most real-world graphs, social networks, maps).

Q: What is the core theoretical difference between Dijkstra's Algorithm and Bellman-Ford?

Theory:

- **Dijkstra:** Uses a **Greedy** approach.[14] It assumes that once a node is "settled" (shortest path found), it can never be improved. This assumption breaks if negative edge weights exist (a "cheaper" path might be found later via a negative edge). Complexity: $O(E + V \log V)$.
- **Bellman-Ford:** Uses **Dynamic Programming** (Relaxation). It relaxes *all* edges $V-1$ times. It can handle negative weights and detect negative cycles. Complexity: $O(VE)$.

VI. Sorting

Q: Why is QuickSort ($O(n^2)$ worst-case) often preferred over MergeSort ($O(n \log n)$ guaranteed)?

Theory:

1. **In-Place:** QuickSort performs partitioning in-place (Space: $O(\log n)$ for stack). MergeSort requires auxiliary space $O(n)$ to merge arrays.
2. **Cache Locality:** QuickSort linearly scans arrays (good spatial locality).[15] MergeSort jumps between subarrays.
3. **Worst Case Rarity:** With randomized pivot selection, the probability of hitting the $O(n^2)$ worst case is statistically negligible.

Q: Is it possible to sort faster than $O(n \log n)$?

Theory:

- **Comparison-Based Sorting:** (Merge, Heap, Quick) requires comparisons.[16] Mathematically, a decision tree of comparisons for n elements has height $\log(n!) \approx n \log n$. You cannot beat this bound.

- **Non-Comparison Sorting:** (Radix, Counting) uses arithmetic properties of keys. They can achieve $O(n)$ or $O(nk)$ but are limited to integers or fixed-length strings (not general purpose).